pid 17 执行 ls：exec 换上新映像（A），到第一个系统调用 open 返回（B）

上：同一进程两个时刻的虚拟地址视角，内核栈按比例画（1 页 = 4096 B）。下：涉及的物理页，箭头落在页内的实际偏移处（实线 = A，虚线 = B）

A · exec 刚换上新映像（exec-image:first）

exec() 已切换页表，还没 return；仍在 sys_exec 的调用链里

struct proc（直接映射区，exec 前后不变）

`0xffffff8006eb7000 = KERNBASE + pa 0x6eb7000`

pid=17 name=ls state=4 RUNNING

vm = 0xffffff8006eb5000　kstack 槽 15

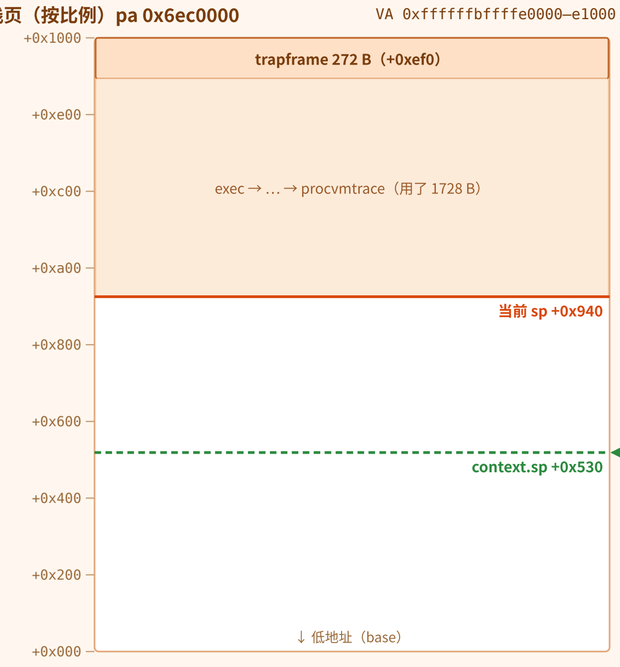
`p->context @ 0xffffff8006eb70b8 (proc + 0xb8)`

`sp = 0xfffffffbffffe0530`

上次 swtch 保存：exec 读盘时睡眠

x19 = 0xfffff80000f9bd0 …（内核现场，x30 = sched 内的返回点）

内核栈页（按比例）pa 0x6ec0000　VA 0xffffffbffffe0000–e1000



trapframe 内容（用户现场）

elr	= 0x2750	ldso_start = 0x2000 + 0x750
spsr	= 0x00000000 （EL0t · 中断开 · 标志清零）	
sp	= 0x8f60	argv 表（exec 刚压好）
x0	= 0x97d0	旧值：return argc 还没写入
x1	= 0x8f60	= sp，argv
x16/x17	= 0x143e0 / 0xe59c	sh 留下的旧值，exec 不清零

用户地址空间（新页表）　sz = 0x9000 · 8 个用户页

TTBR0 页表根　pa 0x6e8c000　libc 尚未装入（sz = 0x9000）

L1[0] → L2　pa 0x6e8a000

L2[0] → L3　pa 0x6e89000

- 程序 /bin/ls　0x0000–0x2000
- ld-xv6.so　0x2000–0x4000　elr 0x2750 → pa 0x6e87750
- 保护页　0x4000–0x5000
- 用户栈 4 页　0x5000–0x9000　sp 0x8f60 → pa 0x6e81f60

B · 第一个系统调用 open 返回前（first-syscall:return）

ld-xv6 调 open("/lib/libc.so")，返回 fd 3

struct proc（直接映射区，exec 前后不变）

`0xffffff8006eb7000 = KERNBASE + pa 0x6eb7000`

pid=17 name=ls state=4 RUNNING

vm = 0xffffff8006eb5000　kstack 槽 15

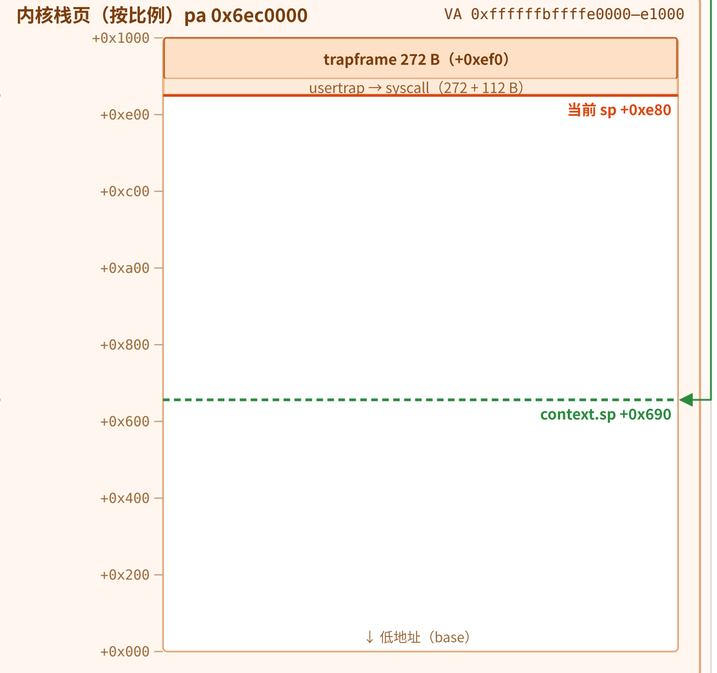
`p->context @ 0xffffff8006eb70b8 (proc + 0xb8)`

`sp = 0xfffffffbffffe0690`

open 在 namei/bread 里睡眠时保存

x19 = 0xfffff80000f9bd0 …（内核现场，x30 = sched 内的返回点）

内核栈页（按比例）pa 0x6ec0000　VA 0xffffffbffffe0000–e1000



trapframe 内容（用户现场）

elr	= 0x2b9c	ld-xv6 的 __syscall(0xb7c)+0x20
spsr	= 0x60000000 （EL0t · 中断开 · Z C）	
sp	= 0x8a30	ld-xv6 的栈帧
x0	= 0x3	open 的返回值：fd 3
x1	= 0x0	
x16/x17	= 0x143e0 / 0xe59c	仍是 sh 留下的旧值

用户地址空间（新页表）　sz = 0x9000 · 8 个用户页

TTBR0 页表根　pa 0x6e8c000　libc 仍未装入：下一步 pread + sbrk

L1[0] → L2　pa 0x6e8a000

L2[0] → L3　pa 0x6e89000

- 程序 /bin/ls　0x0000–0x2000
- ld-xv6.so　0x2000–0x4000　elr 0x2b9c → pa 0x6e87b9c
- 保护页　0x4000–0x5000
- 用户栈 4 页　0x5000–0x9000　sp 0x8a30 → pa 0x6e81a30

从 A 到 B 发生了什么

1. **exec 换页表**
   新根 0x6e8c000；旧 sh 映像随后 uvmfree
   proc、vmspace、内核栈都不变
2. **快照 A**
   还在 sys_exec 里：当前 sp 在 +0x940
   trapframe 已改成 ld-xv6 的入口和新栈
   x0 还是旧值，return argc 之后才写入
3. **回到用户态**
   syscall 写 x0 = argc → usertrapret → eret
   PC = 0x2750（ldso_start），EL0
4. **ld-xv6 调 open**
   svc → el0sync：SP_EL1 = 栈顶 − 272
   在同一 trapframe 处保存现场
5. **open 里睡眠**
   namei → bread 等磁盘 → sleep → sched
   swtch 把 sp 存进 context.sp = +0x690
6. **被唤醒**
   磁盘中断 wakeup → 调度回来
   swtch 恢复 sp，沿调用链返回
7. **快照 B**
   open 返回 3；syscall 正要返回
   当前 sp = +0xe80：只剩 usertrap、syscall 两帧

要点

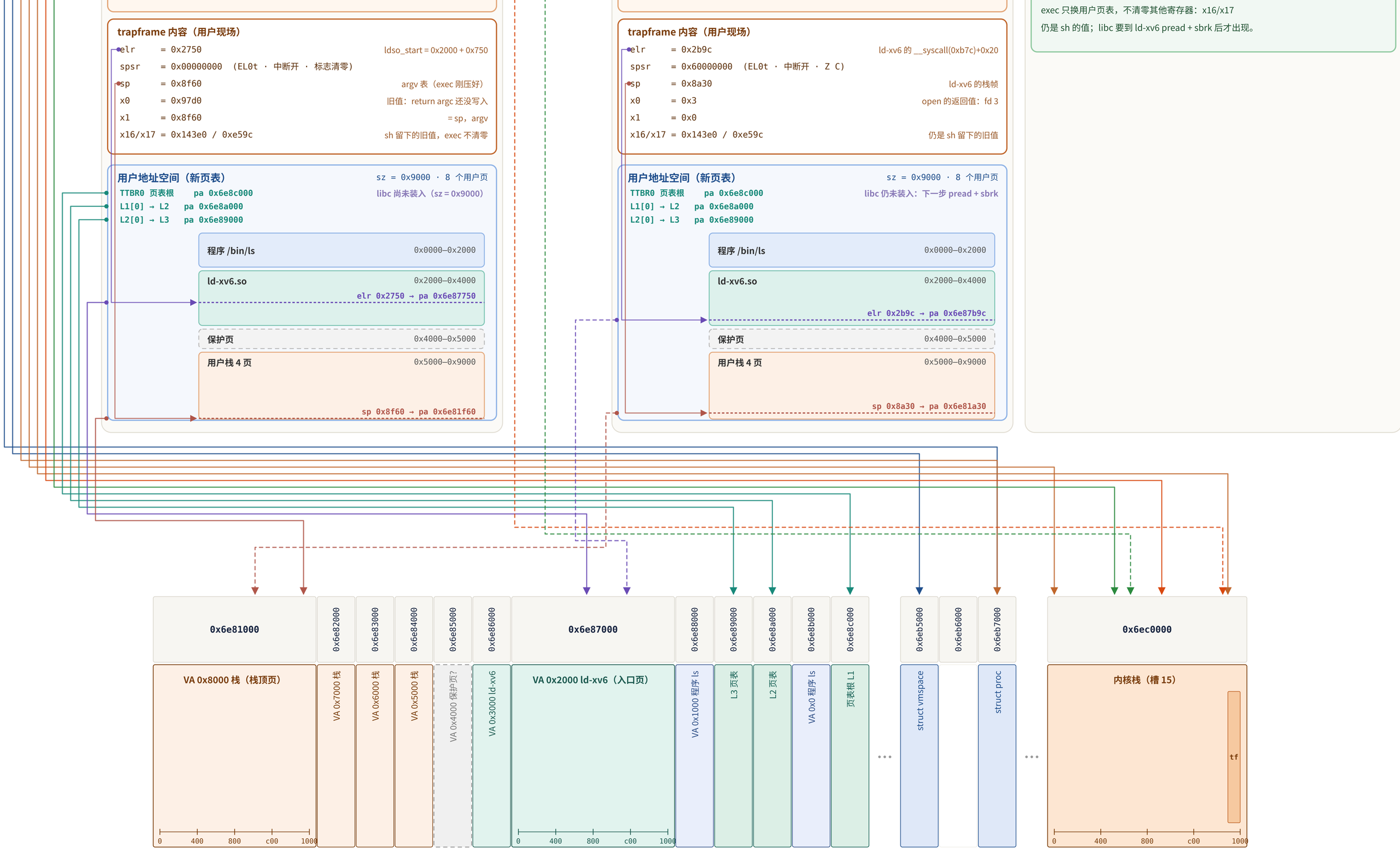
context.sp 只在 swtch 时改写：A 的 +0x530 来自 exec 读 ELF 时的睡眠，B 的 +0x690 来自 open 里的睡眠。

两次都低于 trapframe，深度取决于睡眠时的调用链。

trapframe 始终在 +0xef0：每次 svc 都在原地重写。

当前 sp 是快照那一刻的值：A 在 exec 深处（1728 B），B 只剩 usertrap→syscall（384 B = 272 + 112）。

exec 只换用户页表，不清零其他寄存器：x16/x17 仍是 sh 的值；libc 要到 ld-xv6 pread + sbrk 后才出现。



↑ 涉及的物理页（每列一页；加宽的三列画出页内偏移 0x000 → 0xfff）

struct proc / vmspace（直接映射）　内核栈页、trapframe、p->context　当前内核 sp　context.sp（swtch 保存）　页表　用户 sp　elr

实线 = 快照 A（exec-image:first），虚线 = 快照 B（first-syscall:return）。只有 B 中变化的量（当前 sp、context.sp、用户 sp、elr）从 B 画箭头。

物理页按分配顺序几乎递减：uvmcreate 根 0x6e8c → VA 0 页 0x6e8b → 走页表时分配 L2 0x6e8a、L3 0x6e89 → 后续用户页 0x6e88…0x6e81（0x6e85 推测为保护页）。